

Forward and Backward Building Blocks in Deep Networks

CS4152 — Deep Learning and Neural Networks

Syed Hamed Raza

University of Management and Technology (UMT), Lahore

November 22, 2025

Outline

- 1 Layer-Level Computation
- 2 Backward Propagation
- 3 Network-Level Flow
- 4 Implementation Notes
- 5 Conclusion

Forward Propagation in Layer l

- Inputs: $A^{[l-1]}, W^{[l]}, b^{[l]}$
- Compute:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

- Output:
 - $A^{[l]}$ for next layer
 - Cache containing $Z^{[l]}$ (and optionally $W^{[l]}, b^{[l]}$)

Backward Propagation in Layer l

- Inputs:
 - $dA^{[l]}$ (from layer $l + 1$)
 - Cache: $Z^{[l]}, W^{[l]}, b^{[l]}$
- Compute gradients:

$$dZ^{[l]}, dW^{[l]}, db^{[l]}, dA^{[l-1]}$$

- Outputs:
 - $dA^{[l-1]}$
 - $dW^{[l]}, db^{[l]}$

Full Forward Pass

- Start with $A^{[0]} = X$
- Loop through layers:

$$A^{[1]} \rightarrow A^{[2]} \rightarrow \dots \rightarrow A^{[L]}$$

- Output $\hat{y} = A^{[L]}$
- Store cache from each layer

Full Backward Pass

- Start with:

$$dA^{[L]} = \frac{\partial \mathcal{L}}{\partial A^{[L]}}$$

- Backward sequence:

$$dA^{[L]} \rightarrow dA^{[L-1]} \rightarrow \dots \rightarrow dA^{[1]}$$

- Each layer yields:

$$dW^{[l]}, db^{[l]}, dA^{[l-1]}$$

- Used for gradient descent updates.

Caching Strategy

- Cache stores $Z^{[l]}$ for backward computation.
- In practice, storing $W^{[l]}$ and $b^{[l]}$ inside cache is convenient.
- Avoids passing many parameters through functions.
- Matches structure used in deep learning programming assignments.

Key Takeaways

- Each layer has a forward and backward function.
- Caches connect forward pass to backward pass.
- Repeating these blocks enables arbitrary network depth.
- Provides a modular implementation strategy for deep nets.